

Module MLOps — Projet de Fin d'Apprentissage

PipelineForge

Assistant Scientifique Documentaire (LLMOps)

Conception, industrialisation et déploiement d'un système RAG
appliqué au corpus biomédical PubMed Central Open Access

Réalisé par :

Marouane Daouki
Imane Alouani
Hanane Ayar
Maroua Karroum
El Betti Jihad
Hafsa Elhilali
Chbab Brahim
Bouazza Amina

Encadrant : Pr. Aittdaoud
Équipe PipelineForge — 8 membres
Septembre 2026

Année Universitaire : 2025 – 2026

Résumé

Ce rapport documente la conception, l'implémentation et la validation de la couche d'exposition applicative et de conteneurisation du projet PipelineForge, un assistant scientifique documentaire fondé sur l'architecture RAG (Retrieval-Augmented Generation) appliquée à la littérature biomédicale.

Le projet, mené en équipe de huit personnes sur quatre sprints, a produit une chaîne complète allant de l'ingestion de 700 articles issus de PubMed Central Open Access jusqu'à un service déployé sur une plateforme d'orchestration de conteneurs. Le corpus a été segmenté en 32 146 chunks textuels, validés par une suite de huit tests automatisés, vectorisés au moyen du modèle d'embedding BAAI/bge-base-en-v1.5 en représentations de dimension 768, puis indexés dans une base vectorielle Chroma. La chaîne de génération s'appuie sur un modèle de langage servi par l'API Groq, avec traçabilité complète des expérimentations dans MLflow.

La contribution individuelle documentée ici porte sur le Sprint 4 et consiste à transformer ce module de recherche en un service exploitable : conception d'un contrat d'interface HTTP stable, implémentation d'une API REST au moyen du framework FastAPI avec validation stricte des entrées et sorties par Pydantic, puis conteneurisation de l'ensemble des composants au moyen de Docker et Docker Compose. Cette étape constitue le passage de l'expérimentation à l'exploitation, identifié dans la littérature comme l'un des points de rupture les plus fréquents des projets d'apprentissage automatique.

Les résultats montrent une API fonctionnelle produisant des réponses sourcées avec une latence mesurée d'environ 67 secondes par requête. La validation en conditions réelles, menée en aval par la responsable du déploiement, a néanmoins révélé quatre défauts bloquants dans la livraison initiale, tous imputables à l'absence d'un test de bout en bout préalable à la transmission. Ce constat, analysé en détail dans la section de bilan critique, constitue le principal apport formatif de la mission et illustre concrètement la notion de dette technique cachée décrite par Sculley et ses coauteurs.

Mots-clés : Retrieval-Augmented Generation, LLMOps, MLOps, FastAPI, Docker, conteneurisation, bases vectorielles, PubMed Central, industrialisation de modèles, dette technique.

Abstract

This report documents the design, implementation and validation of the application serving layer and containerisation of PipelineForge, a scientific document assistant based on the Retrieval-Augmented Generation architecture, applied to biomedical literature.

The project produced a complete pipeline, from the ingestion of 700 articles from PubMed Central Open Access to a service deployed on a container orchestration platform. The corpus was segmented into 32,146 text chunks, validated by an automated test suite, embedded using BAAI/bge-base-en-v1.5 into 768-dimensional vectors, and indexed in a Chroma vector database. The individual contribution documented here consists in turning this research module into an operable service: designing a stable HTTP interface contract, implementing a REST API using FastAPI with strict input and output validation through Pydantic, and containerising all components using Docker and Docker Compose.

Results show a functional API producing sourced answers with a measured latency of approximately 67 seconds per request. End-to-end validation, performed downstream by the deployment engineer, nevertheless revealed four blocking defects in the initial delivery, all attributable to the absence of a prior end-to-end test. This finding constitutes the main formative outcome of the mission.

Keywords: Retrieval-Augmented Generation, LLMOps, MLOps, FastAPI, Docker, containerisation, vector databases, PubMed Central, model industrialisation, technical debt.

Remerciements

Je tiens à remercier le Pr. Aittdaoud pour l'encadrement de ce module et pour la structure en sprints imposée au projet, qui a permis de confronter l'équipe à une contrainte de séquençement réaliste plutôt qu'à un travail parallèle artificiel.

Mes remerciements vont également aux sept autres membres de l'équipe PipelineForge. La qualité des documents de passation rédigés par chacun a conditionné directement la possibilité de travailler en fin de chaîne : la précision du README du module RAG rédigé par Hafsa Elhilali, comme celle de la documentation du Vector Store rédigée par El Betti Jihad, ont permis d'éviter des erreurs de conception qui auraient été coûteuses à corriger plus tard.

Je remercie enfin particulièrement Bouazza Amina, dont la rigueur lors de la phase de déploiement a mis en évidence quatre défauts de ma livraison initiale. Ce diagnostic, exigeant et documenté avec précision, a davantage contribué à ma compréhension des exigences réelles de l'industrialisation que ne l'aurait fait une livraison acceptée sans examen.

Table des Matières

Note : pour actualiser ce sommaire, effectuer un clic droit dessus, puis choisir « Mettre à jour les champs » et « Mettre à jour toute la table ».

Liste des Tableaux et Figures

Tableaux

- Tableau 1 — Répartition des rôles techniques au sein de l'équipe PipelineForge
- Tableau 2 — Correspondance entre les composants d'un système RAG et les briques du projet
- Tableau 3 — Comparaison des stratégies de recherche documentaire
- Tableau 4 — Principales bases vectorielles open source
- Tableau 5 — Niveaux de maturité MLOps et positionnement du projet
- Tableau 6 — Calendrier des sprints et livrables collectifs
- Tableau 7 — État du pipeline hérité au début du Sprint 4
- Tableau 8 — Paramètres de vectorisation retenus en amont
- Tableau 9 — Exigences fonctionnelles de la couche d'exposition
- Tableau 10 — Exigences non fonctionnelles
- Tableau 11 — Comparaison des frameworks d'exposition HTTP
- Tableau 12 — Comparaison des serveurs d'application Python
- Tableau 13 — Comparaison des stratégies de conteneurisation
- Tableau 14 — Endpoints exposés par l'API
- Tableau 15 — Schéma de la requête POST /ask
- Tableau 16 — Schéma de la réponse POST /ask
- Tableau 17 — Politique de codes de statut HTTP
- Tableau 18 — Composants déclarés dans le docker-compose
- Tableau 19 — Variables d'environnement du service
- Tableau 20 — Résultats des tests unitaires de l'interface
- Tableau 21 — Défauts identifiés lors du déploiement et corrections apportées
- Tableau 22 — Décomposition de la latence observée
- Tableau 23 — Livrables de la partie FastAPI & Docker
- Tableau 24 — État global de la solution déployée
- Tableau 25 — Synthèse des limites identifiées

Figures

- Figure 1 — Architecture générale du pipeline PipelineForge
- Figure 2 — Schéma de principe d'un système RAG
- Figure 3 — Chaîne de dépendances entre les huit rôles
- Figure 4 — Signature du point d'entrée du module RAG
- Figure 5 — Schémas Pydantic de validation
- Figure 6 — Implémentation de l'endpoint /ask
- Figure 7 — Dockerfile de l'API
- Figure 8 — Extrait du fichier docker-compose.yml
- Figure 9 — Test unitaire de l'interface au moyen d'un substitut
- Figure 10 — Appel réel validé de bout en bout
- Figure 11 — Positionnement de la contribution dans la chaîne de valeur

Glossaire et Abréviations

Terme	Définition
API	Application Programming Interface — interface permettant à un programme d'exposer ses fonctions à d'autres programmes.
ASGI	Asynchronous Server Gateway Interface — norme d'interface entre un serveur web et une application Python asynchrone.
Chunk	Segment de texte issu du découpage d'un document, unité élémentaire d'indexation et de récupération.
CI/CD	Continuous Integration / Continuous Deployment — automatisation de la validation et du déploiement du code.
Cold start	Temps de démarrage initial d'un service, incluant le chargement des modèles en mémoire.
Embedding	Représentation vectorielle dense d'un texte, capturant sa sémantique dans un espace de dimension fixe.
Endpoint	Point d'entrée d'une API, identifié par un chemin et une méthode HTTP.
LLM	Large Language Model — modèle de langage de grande taille.
LLMOps	Ensemble des pratiques d'industrialisation appliquées spécifiquement aux systèmes fondés sur des modèles de langage.
MLOps	Ensemble des pratiques visant à industrialiser le cycle de vie des modèles d'apprentissage automatique.
Parquet	Format de stockage colonnaire optimisé pour les traitements analytiques.
RAG	Retrieval-Augmented Generation — architecture combinant recherche documentaire et génération de texte.
Retrieval	Étape de récupération des documents pertinents pour une requête donnée.
Stub	Substitut logiciel simulant le comportement d'un composant réel, utilisé pour tester un module en isolation.
Top-k	Nombre de documents les plus proches retournés par une recherche vectorielle.
Vector Store	Base de données spécialisée dans le stockage et la recherche de vecteurs par similarité.

1. Introduction

1.1 Contexte général

L'apparition des modèles de langage de grande taille a transformé la manière dont les systèmes informatiques traitent l'information textuelle. Ces modèles, entraînés sur d'immenses corpus, produisent un texte fluide et contextuellement pertinent [1]. Leur adoption dans des domaines exigeants se heurte cependant à deux limitations structurelles bien documentées.

La première tient à la nature même de leur entraînement : les connaissances d'un modèle sont figées à la date de collecte de son corpus. Toute information postérieure lui est inaccessible, ce qui pose un problème particulier dans les domaines scientifiques où la littérature évolue continuellement.

La seconde limitation, plus préoccupante, est le phénomène d'hallucination : la production d'énoncés syntaxiquement corrects et présentés avec assurance, mais factuellement erronés ou entièrement inventés. Ji et ses coauteurs proposent une revue systématique de ce phénomène dans les tâches de génération de langage naturel et soulignent son caractère intrinsèque aux architectures génératives [2]. Dans un contexte biomédical, où une information erronée peut avoir des conséquences sur des décisions cliniques, cette limitation est rédhibitoire si elle n'est pas traitée architecturalement.

L'architecture RAG, proposée par Lewis et ses coauteurs [3], répond directement à ces deux limitations. Son principe consiste à adjoindre au modèle génératif un mécanisme de récupération documentaire : avant de produire une réponse, le système interroge une base de connaissances externe, sélectionne les passages les plus pertinents, et les fournit au modèle comme contexte. La génération devient ainsi conditionnée à des sources vérifiables, et la base peut être mise à jour sans réentraînement.

La génération augmentée par récupération combine une mémoire paramétrique, contenue dans les poids du modèle, avec une mémoire non paramétrique, constituée d'un index de documents consultable à la volée.

Principe formulé par Lewis et al. [3], reformulé

1.2 Problématique

Si les fondements théoriques du RAG sont désormais bien établis et font l'objet de revues de littérature approfondies [4], la question de leur mise en production reste nettement moins traitée. Un prototype RAG fonctionnant dans un carnet de notes interactif et un service RAG exploitable par des utilisateurs réels séparent deux mondes distincts, dont le second concentre l'essentiel des difficultés pratiques.

Sculley et ses coauteurs ont formalisé ce constat dans un article devenu une référence de la discipline : dans un système d'apprentissage automatique réel, le code du modèle proprement dit ne représente qu'une fraction minime de l'ensemble du code nécessaire [5]. La configuration, la collecte de données, la gestion des ressources de calcul, l'infrastructure de service et le monitoring constituent la masse du système, et c'est dans ces composants périphériques que s'accumule ce que les auteurs nomment la dette technique cachée.

La problématique traitée dans ce rapport se formule donc ainsi : comment transformer un module de recherche RAG fonctionnel en un service exploitable, reproductible et déployable, en garantissant que le livrable transmis à l'étape suivante ne nécessite aucune modification pour être mis en production ?

Cette question en recouvre plusieurs, traitées successivement dans ce rapport :

- Quelle forme donner au contrat d'interface entre le module de recherche et le monde extérieur, pour qu'il reste stable malgré les évolutions internes du système ?
- Comment distinguer, dans la gestion des erreurs, une défaillance réelle du service d'une dégradation déjà absorbée par les composants en amont ?

- Quelle stratégie de conteneurisation adopter lorsque les composants du système présentent des natures hétérogènes : service réseau, base de données sur disque, serveur de suivi d'expérimentations ?
- Quel niveau de validation constitue une condition suffisante avant transmission à l'équipe de déploiement ?

1.3 Objectifs

L'objectif général de la mission est d'assurer le passage du module RAG de l'état de prototype à celui de service conteneurisé. Il se décline en objectifs opérationnels :

- **Objectif 1** — Concevoir et implémenter une interface HTTP exposant la fonction de question-réponse du module RAG, avec validation stricte des données échangées.
- **Objectif 2** — Définir une politique de gestion des erreurs cohérente avec le comportement réel des composants en amont, plutôt qu'avec un comportement supposé.
- **Objectif 3** — Conteneuriser l'ensemble des composants nécessaires au fonctionnement du service, en tenant compte de leur hétérogénéité.
- **Objectif 4** — Extraire la configuration sensible du code source et documenter l'ensemble des variables nécessaires au déploiement.
- **Objectif 5** — Produire une documentation technique permettant à la responsable du déploiement de reprendre le livrable sans consultation supplémentaire.

1.4 Méthodologie

La démarche suivie s'articule en cinq phases successives, correspondant aux chapitres 4 à 8 du présent rapport.

Phase d'analyse. Lecture exhaustive des documents de passation des six membres intervenant en amont, extraction des contraintes réelles, et identification de la signature exacte du point d'entrée à exposer. Cette phase, souvent négligée, s'est révélée déterminante : trois décisions de conception majeures découlent directement d'informations qui n'auraient pas été devinées.

Phase de conception. Formalisation des exigences fonctionnelles et non fonctionnelles, comparaison argumentée des options techniques disponibles, définition du contrat d'interface.

Phase d'implémentation. Développement incrémental de l'API, en commençant par un squelette minimal testable avant même la livraison du module RAG, conformément à la recommandation du plan de projet.

Phase de validation. Tests unitaires de l'interface au moyen d'un substitut, puis validation en conditions réelles. C'est sur cette dernière étape que la démarche a présenté une faiblesse, analysée sans complaisance au chapitre 9.

Phase de transmission. Rédaction de la documentation d'intégration et livraison sur le dépôt partagé.

1.5 Organisation du rapport

Le chapitre 2 présente un état de l'art des composants théoriques mobilisés : architecture RAG, représentations vectorielles, bases de données vectorielles, puis pratiques d'industrialisation. Le chapitre 3 décrit le projet PipelineForge, son organisation et son architecture. Le chapitre 4 dresse l'état du pipeline hérité des sprints précédents.

Les chapitres 5 à 8 constituent le cœur de la contribution individuelle : analyse des besoins et conception, implémentation de l'API, conteneurisation, puis validation et résultats. Le chapitre 9 propose un bilan critique, incluant l'analyse des défauts identifiés lors du déploiement. Le chapitre 10 formule des perspectives d'amélioration, et le chapitre 11 conclut.

2. État de l'Art

Ce chapitre présente les fondements théoriques et techniques mobilisés par le projet. Il ne vise pas l'exhaustivité, mais expose les concepts nécessaires à la compréhension des choix documentés dans les chapitres suivants, en les rattachant à la littérature de référence.

2.1 Les modèles de langage et leurs limitations

2.1.1 Architecture Transformer

Les modèles de langage contemporains reposent tous sur l'architecture Transformer, introduite par Vaswani et ses coauteurs [1]. Sa contribution centrale est le mécanisme d'attention, qui permet à chaque élément d'une séquence de pondérer sa relation avec tous les autres éléments, sans recourir à un traitement séquentiel. Cette propriété autorise une parallélisation massive de l'entraînement, condition de la montée en échelle observée depuis.

Devlin et ses coauteurs ont ensuite montré, avec BERT, qu'un pré-entraînement bidirectionnel sur de vastes corpus non annotés produisait des représentations transférables à de nombreuses tâches en aval [6]. Cette approche du pré-entraînement suivi d'un ajustement fin structure encore aujourd'hui l'essentiel du domaine.

2.1.2 Le phénomène d'hallucination

Ji et ses coauteurs proposent une taxonomie des hallucinations en génération de langage naturel, distinguant les hallucinations intrinsèques, contredisant le contexte fourni, des hallucinations extrinsèques, non vérifiables à partir du contexte [2]. Ils analysent les causes de ce phénomène, qui tiennent à la fois aux données d'entraînement et à l'objectif d'apprentissage lui-même, qui optimise la vraisemblance du texte et non sa véracité.

Dans un contexte biomédical, cette distinction a des conséquences directes. Un système produisant une affirmation plausible sur un traitement, sans source vérifiable, est inutilisable quelle que soit la qualité linguistique de sa production. C'est précisément le problème que l'architecture RAG entend traiter.

2.2 L'architecture Retrieval-Augmented Generation

2.2.1 Principe et formulation originale

Lewis et ses coauteurs formalisent le RAG comme la combinaison d'un module de récupération et d'un module de génération, entraînés conjointement [3]. Le module de récupération sélectionne, pour une requête donnée, un ensemble de documents pertinents dans un index externe. Le module génératif produit ensuite une réponse conditionnée à la fois par la requête et par les documents récupérés.

Les auteurs démontrent sur plusieurs tâches à forte intensité de connaissances que cette approche surpasse les modèles purement paramétriques, tout en offrant un avantage pratique décisif : la base de connaissances peut être mise à jour ou remplacée sans réentraînement du modèle.

2.2.2 Évolutions et taxonomie

Gao et ses coauteurs proposent une revue systématique des développements ultérieurs et distinguent trois générations d'architectures [4]. Le RAG naïf enchaîne simplement indexation, récupération et génération. Le RAG avancé introduit des optimisations avant et après la récupération : reformulation de requête, réordonnancement des résultats, compression du contexte. Le RAG modulaire décompose le système en composants interchangeables, permettant des architectures itératives ou adaptatives.

Le système développé dans ce projet relève du RAG naïf, choix assumé compte tenu de la durée du projet et du fait que l'objectif pédagogique porte sur l'industrialisation de la chaîne plutôt que sur l'optimisation de la qualité de récupération.

2.2.3 Correspondance avec les composants du projet

Composant RAG	Réalisation dans PipelineForge	Responsable
Corpus documentaire	700 articles PubMed Central Open Access	Marouane Daouki
Segmentation	32 146 chunks, environ 1200 caractères, recouvrement 200	Imane Alouani
Contrôle qualité	8 tests automatisés, validation Pydantic	Hanane Ayar
Encodeur	BAAI/bge-base-en-v1.5, dimension 768	El Betti Jihad
Index vectoriel	Chroma, collection pipelineforge_pmc	El Betti Jihad
Générateur	Modèle servi par l'API Groq	Hafsa Elhilali
Couche de service	API FastAPI conteneurisée	Chbab Brahim

Tableau 2 — Correspondance entre les composants d'un système RAG et les briques du projet

2.3 La récupération documentaire

2.3.1 Recherche lexicale et recherche dense

La recherche documentaire classique repose sur des méthodes lexicales, dont BM25 constitue la référence. Robertson et Zaragoza en proposent une présentation approfondie [7]. Ces méthodes pondèrent les termes selon leur fréquence dans le document et leur rareté dans le corpus. Leur limite est structurelle : deux formulations sémantiquement équivalentes mais lexicalement disjointes ne se rencontrent pas.

Karpukhin et ses coauteurs ont montré, avec Dense Passage Retrieval, qu'un encodage dense des questions et des passages dans un espace vectoriel commun permettait de dépasser significativement BM25 sur des tâches de question-réponse ouverte [8]. La similarité est alors calculée entre vecteurs, ce qui autorise le rapprochement de formulations différentes exprimant la même idée.

Approche	Principe	Forces	Limites
Lexicale (BM25)	Pondération de la fréquence des termes	Aucun entraînement, interprétable, robuste sur les termes rares	Insensible aux synonymes et reformulations
Dense (DPR, BGE)	Encodage sémantique en vecteurs	Capture la similarité de sens	Coût d'encodage, dépendance au modèle
Hybride	Combinaison des deux scores	Complémentarité des signaux	Complexité d'implémentation et de réglage

Tableau 3 — Comparaison des stratégies de recherche documentaire

Le projet a retenu l'approche dense seule. L'approche hybride, mentionnée comme perspective au chapitre 10, aurait pu améliorer la récupération sur les termes techniques rares, fréquents en biomédecine.

2.3.2 Représentations de phrases

Reimers et Gurevych ont montré que les représentations issues directement de BERT étaient peu adaptées à la comparaison par similarité cosinus, et ont proposé Sentence-BERT, une architecture siamoise produisant

des représentations directement comparables [9]. Cette contribution a ouvert la voie aux modèles d'encodage de phrases utilisés aujourd'hui en production.

Le modèle retenu par le projet, BAAI/bge-base-en-v1.5, s'inscrit dans cette lignée. Xiao et ses coauteurs décrivent la famille BGE et la méthodologie d'entraînement associée dans le cadre du projet C-Pack [10]. Une particularité de ces modèles, documentée par leurs auteurs et correctement appliquée dans le projet, est l'usage d'un préfixe d'instruction du côté de la requête uniquement, et non du côté des documents indexés.

2.4 Les bases de données vectorielles

Le stockage et l'interrogation de grands ensembles de vecteurs constituent un problème algorithmique spécifique. La recherche exacte du plus proche voisin devenant coûteuse en haute dimension, les systèmes pratiques recourent à des méthodes approchées.

Johnson et ses coauteurs décrivent, avec FAISS, une bibliothèque de recherche de similarité optimisée pour le calcul sur processeur graphique, devenue une référence du domaine [11]. Malkov et Yashunin ont proposé l'algorithme HNSW, fondé sur des graphes de navigation hiérarchiques, qui offre un compromis particulièrement favorable entre rappel et latence et équipe aujourd'hui la plupart des bases vectorielles [12].

Système	Nature	Caractéristiques
FAISS	Bibliothèque	Très performante, mais gestion des métadonnées à la charge de l'application
Chroma	Base embarquée	Persistance sur disque, métadonnées natives, aucune administration serveur
Qdrant	Service réseau	Filtrage avancé, conçu pour le déploiement distribué
Milvus	Service distribué	Conçu pour de très grands volumes, exploitation plus lourde

Tableau 4 — Principales bases vectorielles open source

Le choix de Chroma par El Betti Jihad était pertinent au regard du volume traité et de la nécessité de conserver des métadonnées riches auprès de chaque vecteur. Ce choix a néanmoins une conséquence architecturale directe sur la conteneurisation, développée au chapitre 7 : Chroma étant une base embarquée et non un service réseau, elle ne peut être déclarée comme un service dans une composition de conteneurs, mais doit être montée comme un volume.

2.5 L'industrialisation des systèmes d'apprentissage automatique

2.5.1 La dette technique cachée

Sculley et ses coauteurs analysent les mécanismes par lesquels les systèmes d'apprentissage automatique accumulent une dette technique spécifique, distincte de celle des systèmes logiciels classiques [5]. Ils identifient notamment l'enchevêtrement des composants, les dépendances de données non déclarées, et la prolifération du code de liaison entre briques hétérogènes.

La contribution documentée dans ce rapport se situe précisément dans cette zone : le code d'une couche d'exposition ne contient aucun algorithme d'apprentissage, mais conditionne entièrement l'exploitabilité du système. Les quatre défauts identifiés au chapitre 8 relèvent tous de cette catégorie.

2.5.2 Tests et qualité des systèmes ML

Breck et ses coauteurs proposent une grille d'évaluation de la maturité des systèmes d'apprentissage automatique en production, structurée en tests portant sur les données, le modèle, l'infrastructure et le

monitoring [13]. Leur constat central est que les équipes sous-estiment systématiquement les tests d'infrastructure, jugés moins nobles que les tests de modèle.

Amershi et ses coauteurs, à partir d'une étude conduite auprès d'équipes industrielles, aboutissent à un constat convergent et soulignent la difficulté spécifique de la transition entre l'expérimentation et la production [14]. Le présent rapport en fournit une illustration directe.

2.5.3 Niveaux de maturité

Niveau	Caractéristiques	Position du projet
0 — Manuel	Scripts exécutés à la main, aucun enchaînement automatique	Dépassé dès le Sprint 2
1 — Pipeline automatisé	Orchestration des étapes, suivi des expérimentations	Atteint (Prefect, MLflow)
2 — CI/CD complet	Tests, construction et déploiement automatisés, monitoring	Partiellement atteint

Tableau 5 — Niveaux de maturité MLOps et positionnement du projet

Le projet atteint le niveau 1 et une partie du niveau 2 : l'intégration continue est fonctionnelle et le déploiement automatisé, mais le monitoring, dernière composante du niveau 2, n'a pas été mis en place.

2.6 Le service de modèles et la conteneurisation

2.6.1 Architecture REST

Fielding a formalisé le style architectural REST dans sa thèse de doctorat [15], en définissant un ensemble de contraintes dont l'absence d'état côté serveur, l'uniformité de l'interface et l'identification des ressources par des URI. Ces contraintes expliquent la robustesse des interfaces qui les respectent.

Dans le cas d'un service RAG, l'absence d'état revêt une importance particulière : chaque requête doit être traitée indépendamment, sans dépendre d'un état conservé d'une requête à l'autre. Cette propriété conditionne la possibilité de répliquer le service horizontalement.

2.6.2 Conteneurisation

Merkel décrit dans un article fondateur le principe de Docker et l'intérêt des conteneurs pour la reproductibilité des environnements [16]. Contrairement à la virtualisation complète, les conteneurs partagent le noyau du système hôte, ce qui réduit considérablement leur empreinte tout en isolant les dépendances applicatives.

Boettiger analyse spécifiquement l'apport des conteneurs à la reproductibilité de la recherche scientifique et identifie quatre difficultés que Docker permet de traiter : l'enfer des dépendances, la documentation incomplète des environnements, les barrières à l'adoption des codes partagés, et la préservation à long terme [17]. Ces arguments s'appliquent directement à un projet de fin d'apprentissage devant être reproduit par un évaluateur.

Il convient toutefois de noter que la conteneurisation ne garantit la reproductibilité que si les versions des dépendances sont figées. Le quatrième défaut identifié au chapitre 8, un conflit de versions entre deux bibliothèques, illustre précisément ce que la conteneurisation ne résout pas à elle seule.

2.7 L'évaluation des systèmes RAG

Es et ses coauteurs proposent, avec RAGAS, un cadre d'évaluation sans référence pour les systèmes RAG, articulé autour de trois dimensions : la fidélité de la réponse aux documents récupérés, la pertinence de la réponse à la question, et la pertinence des documents récupérés [18]. L'intérêt de ce cadre est de ne pas nécessiter de réponses de référence annotées manuellement, ce qui le rend applicable en conditions réelles.

Chen et ses coauteurs proposent par ailleurs une évaluation comparative des capacités des modèles de langage dans un contexte RAG, portant notamment sur leur robustesse au bruit et leur capacité à refuser de répondre en l'absence d'information pertinente [19]. Cette dernière capacité a été observée qualitativement dans le projet, le modèle signalant explicitement les limites du corpus plutôt que d'inventer une réponse.

Le projet n'a pas mis en place d'évaluation formalisée, limite reconnue par les responsables des Sprints 3 et reprise au chapitre 10 comme perspective prioritaire.

3. Présentation du Projet PipelineForge

3.1 Objectif et périmètre

Le projet PipelineForge vise à construire un assistant scientifique documentaire permettant d'interroger en langage naturel un corpus d'articles scientifiques et d'obtenir une réponse générée, systématiquement adossée aux sources effectivement présentes dans le corpus.

Trois domaines étaient proposés : les matériaux, l'agriculture et la santé. L'équipe a retenu le domaine biomédical lors de l'atelier de cadrage du premier jour, pour une raison essentiellement pragmatique : PubMed Central Open Access expose une interface programmatique riche et bien documentée, ce qui réduisait le risque de blocage sur l'étape d'ingestion, elle-même préalable à toutes les autres.

Ce choix a une conséquence sur la qualité finale du système, observée lors des tests : le corpus de 700 articles, collecté sur des mots-clés multi-thématiques, n'est centré sur aucun sous-domaine particulier. La récupération est de ce fait plus pertinente sur les sujets bien représentés que sur les sujets périphériques.

3.2 Organisation de l'équipe

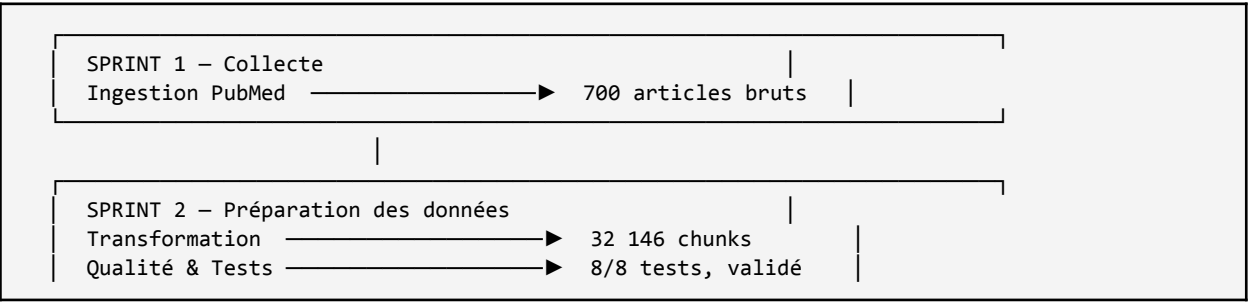
L'équipe compte huit membres occupant huit rôles techniques de statut égal, sans chef de projet. Ce choix organisationnel, imposé par le cadrage du module, repose sur l'idée que la chaîne de dépendances technique remplace la coordination hiérarchique : chaque membre sait précisément de qui il dépend et qui dépend de lui.

#	Membre	Rôle technique	Sprint principal
1	Marouane Daouki	Ingestion de données	Sprint 1
2	Imane Alouani	Transformation de données	Sprint 2
3	Hanane Ayar	Qualité des données et tests	Sprint 2
4	Maroua Karroum	Orchestration	Sprint 2
5	El Betti Jihad	Embeddings et Vector Store	Sprint 3
6	Hafsa Elhilali	Chaîne RAG et expérimentation MLflow	Sprint 3
7	Chbab Brahim	FastAPI et Docker	Sprint 4
8	Bouazza Amina	CI/CD, déploiement et monitoring	Sprint 4

Tableau 1 — Répartition des rôles techniques au sein de l'équipe PipelineForge

3.3 Architecture générale

Le pipeline s'organise en huit étapes séquentielles. Chaque étape consomme le livrable de la précédente et produit un livrable pour la suivante.



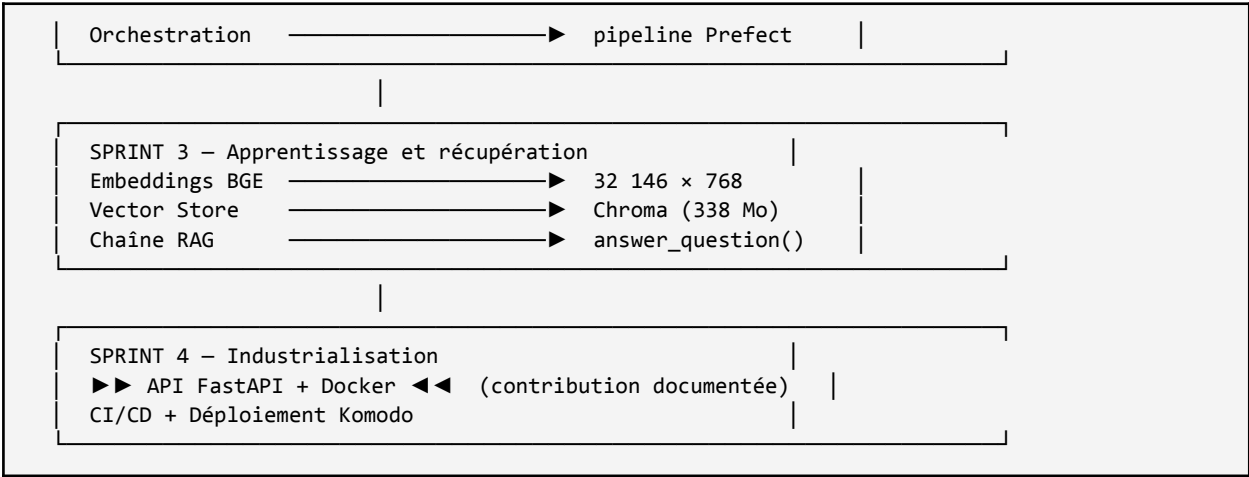


Figure 1 — Architecture générale du pipeline PipelineForge

3.4 Déroulement en sprints

Sprint	Objectifs	Livrable collectif
Sprint 1	Cadrage, stratégie de données, organisation agile, ingestion	Domaine tranché, corpus brut collecté
Sprint 2	Transformation, qualité, tests, orchestration	Pipeline de données automatisé de bout en bout
Sprint 3	Vectorisation, chaîne RAG, suivi des expérimentations	Chaîne RAG fonctionnelle et tracée dans MLflow
Sprint 4	API, conteneurisation, CI/CD, déploiement	Solution déployée et accessible

Tableau 6 — Calendrier des sprints et livrables collectifs

3.5 Chaîne de dépendances

La position en fin de chaîne du rôle documenté ici a une conséquence structurante : le travail ne peut réellement commencer qu'une fois six étapes livrées, ce qui concentre la charge sur une fenêtre temporelle réduite et laisse peu de marge en cas de défaut.

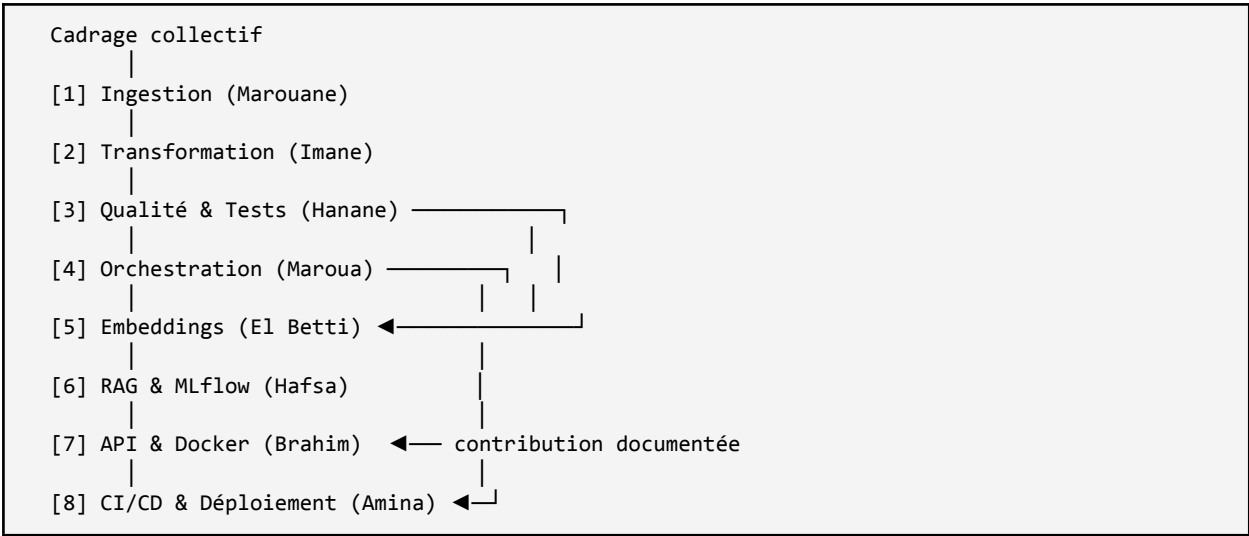


Figure 3 — Chaîne de dépendances entre les huit rôles

3.6 Infrastructure mise à disposition

Le module a mis à disposition de l'équipe un ensemble de ressources demandées collectivement dès le premier jour : un identifiant de groupe, un espace de travail sur le serveur MLOps, un accès à la plateforme de déploiement Komodo, une pile Docker dédiée, une plage de ports réservée, un espace GitHub et un domaine pointant vers le service.

L'usage effectif de ces ressources est traité au chapitre 8 : les droits accordés sur la plateforme de déploiement se sont révélés plus restreints que prévu, ce qui a conduit la responsable du déploiement à adapter sa stratégie.

4. État du Pipeline en Amont de la Mission

Ce chapitre synthétise les livrables des six membres intervenant avant le Sprint 4. Il ne s'agit pas d'une formalité : chaque information présentée ici a directement conditionné une décision de conception documentée dans les chapitres suivants.

4.1 Ingestion des données

Marouane Daouki a développé les scripts de collecte interrogeant les interfaces programmatiques de PubMed Central Open Access. Le livrable comprend 700 articles biomédicaux stockés dans la couche de données brutes, accompagnés d'un index de métadonnées comportant notamment le titre, les auteurs, la date, l'identifiant DOI et la source.

Un élément de ce travail a une incidence directe sur l'API : la présence, dans les métadonnées, d'un champ indiquant si la licence de l'article est restrictive. Ce champ est exploité par le filtre optionnel de la fonction de question-réponse, et se retrouve donc dans le schéma de requête de l'API.

4.2 Transformation

Imane Alouani a pris en charge l'extraction du texte, son nettoyage et sa segmentation. Les 700 articles ont produit 32 146 chunks, stockés au format Parquet.

Le paramétrage retenu est une segmentation d'environ 1200 caractères avec un recouvrement de 200 caractères. Ce calibrage en caractères, et non en unités lexicales, constitue une approximation dont les conséquences ont été mesurées ultérieurement lors de la vectorisation.

4.3 Qualité des données et tests

Hanane Ayar a implémenté huit tests automatisés au moyen de pytest, complétés par un script de validation combinant pandas et Pydantic. Les tests portent sur la conformité du schéma, l'absence de valeurs nulles, les contraintes de longueur, la couverture des documents sources, l'encodage, la détection des doublons, l'intégrité du fichier Parquet et le référencement croisé avec les métadonnées.

Deux décisions analytiques issues de ce travail méritent d'être signalées : l'acceptation d'une proportion de chunks sans titre de section explicite, au motif que le contenu sémantique prévaut sur la structure en recherche vectorielle, et la suppression stricte des sections non informatives telles que les références bibliographiques et les remerciements, afin de réduire le bruit dans l'index.

Le choix de Pydantic par Hanane Ayar présente un intérêt qui dépasse son périmètre : la même bibliothèque a pu être réutilisée au Sprint 4 pour la validation des entrées et sorties de l'API, sans introduire de nouvelle dépendance conceptuelle dans le projet.

4.4 Orchestration

Maroua Karroum a implémenté un pipeline d'orchestration au moyen de Prefect, enchaînant en six tâches la vérification des données brutes, la transformation, l'exécution des tests, la validation qualité, la vérification de la sortie et la confirmation de transmission.

L'ensemble s'exécute par une commande unique, avec journalisation structurée des transitions d'état et mécanisme de reprise sur erreur. Cette propriété s'est révélée précieuse en aval : plusieurs membres ont dû régénérer localement l'intégralité de la chaîne de données, celle-ci n'étant pas versionnée.

4.5 Vectorisation et Vector Store

El Betti Jihad a sélectionné le modèle BAAI/bge-base-en-v1.5, retenu pour son fonctionnement local sans dépendance à un service externe et pour la qualité de ses représentations. Une vérification de calibration menée sur un échantillon de 500 chunks a mis en évidence que le découpage en caractères produisait une longueur moyenne d'environ 248 unités lexicales, avec un maximum observé de 911.

Paramètre	Valeur
Modèle d'encodage	BAAI/bge-base-en-v1.5
Dimension des vecteurs	768
Fenêtre de contexte du modèle	512 unités lexicales
Longueur moyenne des chunks	environ 248 unités lexicales
Chunks excédant la fenêtre	4,0 % (environ 1 286 sur 32 146)
Normalisation	Activée, similarité cosinus
Durée de génération	76 minutes
Base vectorielle	Chroma, collection pipelineforge_pmc
Volume sur disque	environ 338 Mo

Tableau 8 — Paramètres de vectorisation retenus en amont

La décision de tronquer les 4 % de chunks excédant la fenêtre du modèle, plutôt que de régénérer l'intégralité du corpus avec un paramétrage réduit, a été documentée et assumée. Elle illustre un arbitrage courant en ingénierie : accepter une dégradation mesurée et circonscrite plutôt que de bloquer l'avancement d'une chaîne séquentielle.

Point déterminant pour le Sprint 4 : le Vector Store est une base embarquée persistant sur disque, et non un service réseau. Cette caractéristique, clairement documentée dans le README du module, a été initialement mal interprétée lors de la conception de la conteneurisation. L'erreur qui en a résulté est analysée au chapitre 8.

4.6 Chaîne RAG et suivi des expérimentations

Hafsa Elhilali a construit la chaîne complète et structuré le module en quatre fichiers : un module de récupération chargeant le Vector Store et encodant les questions, un module de génération construisant le prompt et appelant le modèle de langage, un point d'entrée unique intégrant le suivi des expérimentations, et un jeu de questions de test.

Le modèle de langage retenu est servi par l'API Groq, choix motivé par la gratuité de l'accès en phase de développement, la faible latence et l'absence de dépendance aux ressources locales de la machine.

Interface exposée

```
def answer_question(question: str,
                    top_k: int = 5,
                    where: dict = None) -> dict:
    """
    Répond à une question en s'appuyant sur le corpus indexé.
    Trace l'appel dans MLflow (paramètres, métriques, durées).

    Retourne :
    {
        "answer": str,
        "sources": [
```

```
        {"title": str, "section": str, "distance": float},
        ...
    ]
}
"""
```

Figure 4 — Signature du point d'entrée du module RAG

Trois contraintes héritées

La lecture du code et du README de ce module a fait apparaître trois contraintes qui ont directement structuré la conception de l'API.

Première contrainte. Les défaillances du service de génération sont déjà traitées à l'intérieur du module. Un bloc de gestion d'exception entoure l'appel au modèle de langage et produit, en cas d'échec, un message de repli inséré dans le champ de réponse, l'erreur étant par ailleurs journalisée dans le suivi d'expérimentations. Aucune exception ne remonte donc jusqu'à la couche d'exposition pour ce cas de figure. Cette information a déterminé la politique de codes de statut HTTP documentée au chapitre 6.

Deuxième contrainte. Chaque appel ouvre automatiquement une session de suivi d'expérimentation. Un serveur de suivi accessible constitue donc une dépendance obligatoire du service, et non une facilité optionnelle. Cette contrainte a imposé la déclaration d'un second conteneur dans la composition.

Troisième contrainte. Le modèle d'encodage est téléchargé au premier appel s'il n'est pas déjà présent en cache. Le conteneur doit donc disposer d'un accès réseau sortant au démarrage, point explicitement signalé par l'auteure du module à l'attention de l'équipe de déploiement.

4.7 Synthèse de l'état hérité

Indicateur	Valeur	Source
Articles ingérés	700	Marouane Daouki
Chunks générés	32 146	Imane Alouani
Tests qualité réussis	8 sur 8	Hanane Ayar
Pipeline orchestré	6 tâches, exécution en une commande	Maroua Karroum
Vecteurs indexés	32 146, dimension 768	El Betti Jihad
Volume du Vector Store	environ 338 Mo	El Betti Jihad
Questions RAG validées	8 sur 8, réponses sourcées	Hafsa Elhilali
Meilleure distance de récupération	0,422	Hafsa Elhilali

Tableau 7 — État du pipeline hérité au début du Sprint 4

Une particularité doit être signalée car elle a eu une incidence pratique majeure : ni le corpus validé, ni les vecteurs, ni le Vector Store ne sont versionnés sur le dépôt, en raison de leur volume. Chaque membre doit donc les régénérer localement, opération demandant entre quatre-vingts et quatre-vingt-dix minutes. Cette contrainte a directement pesé sur la décision, analysée au chapitre 9, de ne pas mener de test de bout en bout avant transmission.

5. Analyse des Besoins et Conception

5.1 Positionnement de la contribution

La contribution documentée occupe une position particulière dans la chaîne : elle ne produit ni donnée, ni modèle, ni résultat scientifique. Elle produit une frontière. Avant elle, le système est un ensemble de modules Python exécutables par leur auteur ; après elle, il devient un service interrogeable par un tiers.

Cette transition est identifiée dans la littérature comme l'une des plus critiques du cycle de vie d'un système d'apprentissage automatique. Amershi et ses coauteurs relèvent que les équipes sous-estiment régulièrement l'effort nécessaire, précisément parce que le travail ne produit aucun résultat visible sur les métriques du modèle [14].

AVANT		APRÈS
<code>python rag/rag_chain.py</code>	▶▶▶	<code>POST /ask</code> <code>{ "question": "..."} </code>
Exécutable par son auteur		Interrogeable par tout client
Dépendances implicites		Environnement déclaré
Configuration en dur		Configuration externalisée
Une machine		Tout hôte disposant de Docker

Figure 11 — Positionnement de la contribution dans la chaîne de valeur

5.2 Exigences fonctionnelles

Réf.	Exigence	Priorité
EF1	Exposer la fonction de question-réponse via une interface HTTP	Critique
EF2	Accepter les paramètres optionnels de la fonction sous-jacente (nombre de résultats, filtre de métadonnées)	Élevée
EF3	Retourner la réponse accompagnée de la liste complète des sources utilisées	Critique
EF4	Fournir un point de vérification de disponibilité indépendant des dépendances lourdes	Critique
EF5	Valider les données entrantes et rejeter explicitement les requêtes malformées	Élevée
EF6	Exposer une documentation d'interface consultable	Moyenne
EF7	Mesurer et retourner la latence de traitement de chaque requête	Moyenne

Tableau 9 — Exigences fonctionnelles de la couche d'exposition

5.3 Exigences non fonctionnelles

Réf.	Exigence	Justification
ENF1	Reproductibilité de l'environnement d'exécution	Condition du déploiement sur un hôte tiers
ENF2	Absence de secret dans le code source	Le dépôt est partagé entre huit personnes
ENF3	Absence d'état conservé entre les requêtes	Condition de la réplication horizontale [15]
ENF4	Stabilité du contrat d'interface	Toute évolution impacte l'équipe de déploiement
ENF5	Documentation suffisante pour une reprise autonome	Critère explicite de la définition de fin de mission
ENF6	Traçabilité des erreurs par journalisation	Nécessaire au diagnostic en production

Tableau 10 — Exigences non fonctionnelles

5.4 Choix du framework d'exposition

5.4.1 Options considérées

Framework	Caractéristiques	Décision
Flask	Micro-framework très répandu, écosystème mature. Validation des données à implémenter manuellement, documentation d'interface non générée, support asynchrone limité.	Non retenu
Django REST Framework	Solution complète intégrant authentification, permissions et sérialisation. Suppose l'adoption du modèle de données Django, hors de proportion pour deux points d'entrée.	Non retenu
FastAPI	Validation native par Pydantic, documentation OpenAPI générée automatiquement, support asynchrone natif, performances élevées grâce à Starlette.	Retenu

Tableau 11 — Comparaison des frameworks d'exposition HTTP

5.4.2 Justification du choix

Trois arguments ont fondé la sélection de FastAPI, par ordre d'importance décroissante.

Cohérence avec les choix de l'équipe. Pydantic ayant déjà été retenu par Hanane Ayar au Sprint 2 pour la validation du corpus, son emploi pour la validation de l'interface ne constitue pas une dépendance supplémentaire mais une extension d'un outil déjà maîtrisé collectivement. Cet argument, qui relève de la cohérence d'équipe plus que de la performance technique, a pesé de manière décisive.

Réduction de la surface d'erreur. La validation déclarative supprime une catégorie entière de défauts : une requête malformée est rejetée par le framework avant d'atteindre le code applicatif, avec un message d'erreur structuré indiquant précisément le champ fautif. Cette propriété est particulièrement précieuse lorsque le consommateur de l'interface est une autre personne de l'équipe.

Documentation auto-générée. La génération automatique d'une documentation d'interface conforme à la spécification OpenAPI fournit à l'équipe de déploiement un moyen d'explorer et de tester l'interface sans lire le code. Cet apport réduit le coût de transmission entre deux personnes.

5.5 Choix du serveur d'application

Serveur	Nature	Adéquation
Gunicorn	WSGI, synchrone, multi-processus	Incompatible avec une application ASGI sans adaptateur
Uvicorn	ASGI, fondé sur uvloop	Adapté nativement, recommandé par la documentation de FastAPI
Hypercorn	ASGI, support HTTP/2	Fonctionnalités additionnelles non requises ici

Tableau 12 — Comparaison des serveurs d'application Python

Uvicorn a été retenu comme serveur d'exécution. Il convient de noter que la configuration mise en place n'exploite qu'un unique processus de travail : compte tenu de la latence observée, une configuration multi-processus aurait pu améliorer le débit, point repris au chapitre 10.

5.6 Choix de la stratégie de conteneurisation

Approche	Caractéristiques	Décision
----------	------------------	----------

Image unique	Simplicité maximale. Ne permet pas de faire coexister l'API et le serveur de suivi d'expérimentations, dépendance obligatoire du module RAG.	Insuffisant
Kubernetes	Orchestration avancée, mise à l'échelle automatique. Hors de proportion pour deux services, et incompatible avec les droits disponibles sur l'infrastructure du module.	Non retenu
Docker Compose	Description déclarative de plusieurs services, gestion des volumes et des variables d'environnement, exploitable directement par la plateforme de déploiement retenue par l'équipe.	Retenu

Tableau 13 — Comparaison des stratégies de conteneurisation

Un argument supplémentaire, découvert lors de l'analyse du contexte de déploiement, a confirmé ce choix : la plateforme Komodo utilisée par l'équipe accepte directement un fichier de composition comme définition d'une pile de services. Produire ce format constituait donc l'interface la plus directe avec l'étape suivante.

5.7 Architecture cible retenue

L'architecture retenue comporte deux conteneurs et un volume monté, cette dernière particularité découlant de la nature embarquée de la base vectorielle.

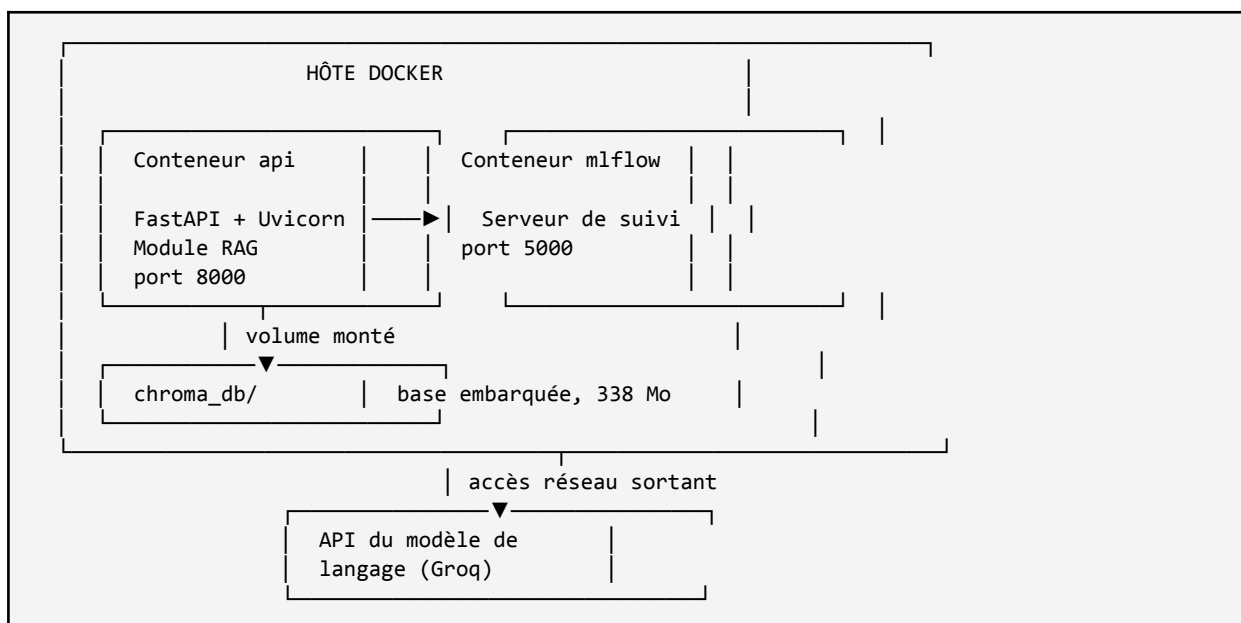


Figure 8 — Architecture cible de la solution conteneurisée

6. Implémentation de l'Interface Applicative

6.1 Organisation du code

Le code de la couche d'exposition est regroupé dans un répertoire dédié, conformément à la convention adoptée par l'équipe, chaque rôle disposant de son propre répertoire à la racine du dépôt.

api/	
├─ main.py	Application, déclaration des points d'entrée
├─ schemas.py	Modèles de validation des données échangées
├─ requirements.txt	Dépendances de la couche d'exposition
├─ Dockerfile	Définition de l'image
└─ README.md	Documentation d'intégration

6.2 Définition du contrat d'interface

6.2.1 Points d'entrée

Chemin	Méthode	Rôle	Dépendances externes
/health	GET	Vérification de disponibilité	Aucune
/ask	POST	Question vers réponse sourcée	Vector Store, modèle de langage, serveur de suivi
/docs	GET	Documentation interactive générée	Aucune

Tableau 14 — Endpoints exposés par l'API

La séparation entre un point de vérification de disponibilité et un point métier constitue une décision de conception délibérée. Un système de supervision ou un orchestrateur doit pouvoir vérifier qu'un conteneur est vivant sans déclencher un traitement coûteux. Compte tenu de la latence mesurée en conditions réelles, utiliser le point métier à cette fin aurait rendu toute supervision périodique impraticable.

6.2.2 Schéma de requête

Champ	Type	Obligatoire	Contrainte	Rôle
question	chaîne	Oui	longueur minimale 1	Question de l'utilisateur
top_k	entier	Non	entre 1 et 50, défaut 5	Nombre de chunks récupérés
where	objet	Non	aucune	Filtre sur les métadonnées

Tableau 15 — Schéma de la requête POST /ask

Les bornes imposées au nombre de résultats méritent une justification. La borne inférieure interdit une valeur nulle ou négative, qui provoquerait une erreur dans la couche de récupération. La borne supérieure prévient une saturation de la fenêtre de contexte du modèle de langage : un nombre excessif de chunks produirait un prompt dépassant les limites acceptées et un échec de génération. Cette contrainte, déclarée au niveau de la validation, remplace un contrôle qu'il aurait fallu écrire manuellement.

6.2.3 Schéma de réponse

Champ	Type	Origine
answer	chaîne	Produit par le module RAG
sources	liste d'objets	Produit par le module RAG

<code>sources[].title</code>	chaîne	Métadonnée du chunk
<code>sources[].section</code>	chaîne	Métadonnée du chunk
<code>sources[].distance</code>	décimal	Distance de récupération
<code>latency_ms</code>	décimal	Mesuré par la couche d'exposition

Tableau 16 — Schéma de la réponse *POST /ask*

Le schéma de réponse reproduit exactement la structure retournée par la fonction sous-jacente, augmentée d'un unique champ mesuré par la couche d'exposition. Ce choix est délibéré : introduire une transformation intermédiaire aurait créé un point de désynchronisation potentiel entre les deux modules, et aurait masqué toute évolution du contrat en amont au lieu de la révéler.

6.3 Implémentation des modèles de validation

```
from pydantic import BaseModel, Field
from typing import List, Optional, Dict

class AskRequest(BaseModel):
    question: str = Field(..., min_length=1,
                           description="Question de l'utilisateur")
    top_k: Optional[int] = Field(5, ge=1, le=50,
                                 description="Nombre de chunks à récupérer")
    where: Optional[Dict] = Field(None,
                                   description="Filtre sur les métadonnées")

class Source(BaseModel):
    title: Optional[str] = None
    section: Optional[str] = None
    distance: Optional[float] = None

class AskResponse(BaseModel):
    answer: str
    sources: List[Source]
    latency_ms: float

class HealthResponse(BaseModel):
    status: str
```

Figure 5 — Schémas Pydantic de validation

Les champs du modèle représentant une source sont déclarés optionnels. Cette permissivité apparente répond à une contrainte réelle : les métadonnées des articles ne sont pas uniformément renseignées, certains chunks ne disposant pas d'un titre de section explicite, comme documenté par la responsable du contrôle qualité. Imposer un champ obligatoire aurait provoqué un échec de sérialisation sur une réponse par ailleurs parfaitement valide.

6.4 Politique de gestion des erreurs

6.4.1 Analyse préalable du comportement réel

La conception de la gestion des erreurs a reposé sur une lecture du code du module en amont plutôt que sur une hypothèse. Cette lecture a révélé que l'appel au modèle de langage est entouré d'un bloc de gestion d'exception produisant, en cas d'échec, une réponse de repli explicite.

La conséquence est contre-intuitive : une indisponibilité du service de génération ne produit pas une erreur, mais une réponse valide au format attendu, dont le contenu signale la dégradation. Propager systématiquement un code d'erreur serveur dans ce cas aurait été doublement incorrect, en signalant une défaillance du service qui n'a pas lieu, et en privant le client d'une réponse qu'il pouvait exploiter.

6.4.2 Politique retenue

Situation	Code	Corps de réponse
Requête valide, traitement nominal	200	Réponse complète avec sources
Requête valide, service de génération indisponible	200	Réponse contenant le message de repli
Champ manquant ou type incorrect	422	Détail de l'erreur de validation
Vector Store introuvable ou inaccessible	500	Message d'erreur interne
Serveur de suivi d'expérimentations inaccessible	500	Message d'erreur interne

Tableau 17 — Politique de codes de statut HTTP

6.4.3 Implémentation

```
@app.post("/ask", response_model=AskResponse)
def ask(request: AskRequest):
    """
    Note : answer_question() ne lève pas d'exception en cas de panne
    du modèle de langage – generation.py gère déjà ce cas en interne
    et renvoie un message de repli. Une exception ici signale donc un
    problème plus grave (Vector Store, MLflow, import du module).
    """
    start = time.time()
    try:
        result = answer_question(
            question=request.question,
            top_k=request.top_k,
            where=request.where,
        )
    except Exception as e:
        logger.exception("Échec de answer_question()")
        raise HTTPException(
            status_code=500,
            detail=f"Erreur interne du pipeline RAG : {e}",
        )

    latency_ms = (time.time() - start) * 1000

    return AskResponse(
        answer=result["answer"],
        sources=[Source(**s) for s in result["sources"]],
        latency_ms=latency_ms,
    )
```

Figure 6 — Implémentation de l'endpoint /ask

Le commentaire inséré en tête de la fonction n'est pas décoratif : il documente une décision de conception non évidente à la lecture du code seul, et évite qu'une future modification ne restaure par erreur une gestion

d'exception inutile. Cette pratique répond directement à la problématique du code de liaison identifiée par Sculley et ses coauteurs, dont l'opacité constitue une forme de dette [5].

6.5 Journalisation

La journalisation des exceptions emploie une méthode conservant la trace d'appel complète. En production, une erreur signalée sans son contexte d'apparition impose de reproduire le problème pour le diagnostiquer, ce qui peut être coûteux lorsque la reproduction suppose la régénération d'un environnement de plusieurs centaines de mégaoctets.

Cette journalisation reste néanmoins rudimentaire au regard des recommandations de Breck et ses coauteurs, qui préconisent une instrumentation permettant de suivre la distribution des entrées et la dérive des performances dans le temps [13]. Cette limite est reprise au chapitre 10.

7. Conteneurisation de la Solution

7.1 Enjeux

La conteneurisation répond aux difficultés de reproductibilité analysées par Boettiger [17] : enfer des dépendances, documentation incomplète des environnements, et impossibilité de rejouer une exécution à distance de sa production. Dans le cadre du projet, ces difficultés ne sont pas théoriques. Le pipeline mobilise des bibliothèques d'apprentissage profond volumineuses, un client de base vectorielle, un client de service de langage et un serveur de suivi d'expérimentations, chacun avec ses propres contraintes de version.

Trois particularités du système compliquent l'exercice au-delà d'un cas standard. La base vectorielle est embarquée et non un service réseau. Le serveur de suivi d'expérimentations est une dépendance obligatoire et non un outil de développement. Le modèle d'encodage est téléchargé au premier appel, ce qui introduit une dépendance réseau au démarrage.

7.2 Construction de l'image applicative

7.2.1 Contexte de construction

Une contrainte structurante doit être posée d'emblée : la couche d'exposition importe directement les fonctions du module RAG. Ce dernier doit donc être présent dans le contexte de construction de l'image. En conséquence, la construction s'effectue depuis la racine du dépôt et non depuis le répertoire de l'API, ce qui constitue une inversion par rapport à l'organisation apparente du code.

```
# Dockerfile – API FastAPI
# Construit depuis la racine du dépôt afin de copier api/ ET rag/
# Commande : docker build -f api/Dockerfile -t pipelineforge-api .

FROM python:3.11-slim

WORKDIR /app

# --- Dépendances de la couche d'exposition ---
COPY api/requirements.txt ./requirements-api.txt
RUN pip install --no-cache-dir -r requirements-api.txt

# --- Code de la couche d'exposition ---
COPY api/ ./

# --- Module RAG (retrieval, génération, chaîne) ---
COPY rag/ ./rag_module/
ENV PYTHONPATH="/app/rag_module:${PYTHONPATH}"

EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Figure 7 — Dockerfile de l'API

7.2.2 Justification des choix

Image de base allégée. La variante réduite de l'image Python officielle a été retenue afin de limiter la taille finale et la surface d'attaque. Ce choix suppose que les dépendances disposent de distributions précompilées pour cette variante, condition vérifiée pour l'ensemble des bibliothèques employées.

Ordre des instructions. Le fichier de dépendances est copié et installé avant le code applicatif. Ce séquençement exploite le mécanisme de cache par couches : une modification du code ne réinvalide pas l'installation des dépendances, ce qui réduit fortement la durée des constructions successives.

Absence de cache d'installation. L'option supprimant le cache du gestionnaire de paquets évite d'embarquer des archives inutiles dans l'image finale.

Adresse d'écoute. Le serveur écoute sur toutes les interfaces et non sur l'interface locale seule. Cette configuration est indispensable en environnement conteneurisé : un service écoutant uniquement sur l'interface locale du conteneur serait inaccessible depuis l'hôte, défaut classique et difficile à diagnostiquer.

7.2.3 Limite initiale de cette définition

La version du fichier présentée ci-dessus est la version corrigée. La version initialement transmise comportait deux défauts, analysés au chapitre 8 : une erreur de syntaxe provoquée par un commentaire placé sur la même ligne qu'une commande, et l'absence d'installation des dépendances propres au module RAG, laissées en commentaire dans l'attente d'une vérification jamais effectuée.

7.3 Composition des services

7.3.1 Définition

```
services:
  api:
    build:
      context: .
      dockerfile: api/Dockerfile
    ports:
      - "8000:8000"
    env_file:
      - .env
    environment:
      - MLFLOW_TRACKING_URI=http://mlflow:5000
    volumes:
      # Vector store Chroma : base locale sur disque,
      # PAS un service réseau. Doit exister avant le lancement.
      - ./embeddings/output/chroma_db:/app/embeddings/output/chroma_db
    depends_on:
      - mlflow

  mlflow:
    image: ghcr.io/mlflow/mlflow:v2.19.0
    ports:
      - "5000:5000"
    volumes:
      - mlflow_data:/mlflow
    command: >
      mlflow server
      --host 0.0.0.0
      --port 5000
      --backend-store-uri /mlflow/mlflow.db
      --default-artifact-root /mlflow/artifacts

volumes:
  mlflow_data:
```

Figure 8 — Extrait du fichier docker-compose.yml

7.3.2 Composants déclarés

Composant	Nature	Port	Persistance
api	Image construite localement	8000	Aucune (sans état)
mlflow	Image officielle	5000	Volume nommé
chroma_db	Volume monté depuis l'hôte	—	Répertoire hôte, environ 338 Mo

Tableau 18 — Composants déclarés dans le docker-compose

7.3.3 Le traitement du Vector Store

Le point le plus délicat de cette configuration concerne la base vectorielle, et c'est également celui sur lequel la conception initiale s'est trompée.

Chroma est une base embarquée : elle ne s'exécute pas comme un processus indépendant écoutant sur un port, mais comme une bibliothèque manipulant des fichiers sur disque. Elle ne peut donc pas être déclarée comme un service dans une composition de conteneurs. Elle doit être rendue accessible au conteneur applicatif par un montage de volume.

La version initialement transmise comportait sur ce point deux erreurs cumulées : un service de base vectorielle en réseau était déclaré, alors qu'aucun composant du système ne l'interroge ; et le volume contenant la base réellement utilisée n'était pas monté. Un troisième défaut s'y ajoutait, le montage ayant ensuite été réalisé en lecture seule alors que la bibliothèque requiert un accès en écriture, y compris pour des opérations de consultation.

Le montage en lecture seule d'une base embarquée constitue une erreur de raisonnement compréhensible mais coûteuse : l'intuition suggère qu'une opération de lecture n'exige qu'un accès en lecture, alors que le moteur sous-jacent écrit des fichiers de journalisation et de verrouillage y compris lors d'une simple consultation.

Analyse du troisième défaut identifié

7.4 Externalisation de la configuration

Conformément aux principes de l'application à douze facteurs, la configuration dépendante de l'environnement est extraite du code et injectée à l'exécution. Cette pratique répond à une exigence non fonctionnelle explicite : le dépôt étant partagé entre huit personnes, un secret inscrit dans le code y serait exposé de façon permanente, y compris dans l'historique des versions.

Variable	Rôle	Consommée par	Sensible
GROQ_API_KEY	Authentification auprès du service de génération	Module de génération	Oui
MLFLOW_TRACKING_URI	Adresse du serveur de suivi	Chaîne RAG	Non
VECTOR_DB_URL	Convention d'équipe, chemin de la base	Non consommée directement	Non

Tableau 19 — Variables d'environnement du service

Seul un fichier modèle, dépourvu de valeurs réelles, est versionné. Un point mérite d'être relevé sur la méthode : un fichier modèle existait déjà, créé par un autre membre et déclarant deux variables. Plutôt que de l'écraser par la version issue de ce travail, les deux fichiers ont été fusionnés manuellement. Ce détail, en apparence mineur, illustre une exigence du travail collaboratif : un fichier partagé n'appartient à personne, et l'écraser revient à supprimer silencieusement le travail d'autrui.

7.5 Ce que la conteneurisation ne résout pas

Il convient de nuancer l'apport de la conteneurisation. Celle-ci garantit que le même environnement sera reproduit à l'identique, mais ne garantit pas que cet environnement soit cohérent.

Le quatrième défaut identifié lors du déploiement en fournit une illustration nette : un conflit entre deux bibliothèques exigeant des versions incompatibles d'une même dépendance transitive. Ce conflit se serait reproduit identiquement sur toute machine, précisément parce que l'environnement était correctement reproductible. La conteneurisation a donc fidèlement reproduit un défaut, au lieu de le masquer derrière une différence d'environnement — ce qui constitue en soi une propriété souhaitable, mais souligne que reproductibilité ne signifie pas correction.

8. Validation et Résultats

8.1 Stratégie de validation

Deux niveaux de validation étaient envisageables. Le premier, en isolation, teste le contrat d'interface au moyen d'un substitut simulant le module RAG. Le second, en conditions réelles, exécute la chaîne complète avec le Vector Store, le service de génération et le serveur de suivi.

Seul le premier niveau a été mené avant transmission. Cette décision, ses justifications et ses conséquences font l'objet d'une analyse détaillée au chapitre 9.

8.2 Validation en isolation

8.2.1 Méthode

Un substitut reproduisant la signature et le format de retour de la fonction sous-jacente a été inséré à la place du module réel. Cette technique permet de vérifier l'intégralité du contrat HTTP sans dépendre d'un environnement dont la mise en place demande plus d'une heure.

```
# Substitut simulant answer_question()
def answer_question(question, top_k=5, where=None):
    return {
        "answer": f"Réponse simulée à : {question}",
        "sources": [
            {"title": "Article Test",
             "section": "Résultats",
             "distance": 0.12}
        ],
    }

# Test de l'interface
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

r1 = client.get("/health")
# -> 200 {"status": "ok"}

r2 = client.post("/ask", json={"question": "Comment le diabète
                               est-il diagnostiqué ?",
                               "top_k": 3})
# -> 200 {answer, sources, latency_ms}
```

Figure 9 — Test unitaire de l'interface au moyen d'un substitut

8.2.2 Résultats

Test	Résultat attendu	Observé
Chargement de l'application	Import sans exception	Conforme
GET /health	Code 200, statut nominal	Conforme
POST /ask avec charge valide	Code 200, structure complète	Conforme
POST /ask sans le champ question	Code 422, erreur de validation	Conforme
POST /ask avec nombre de résultats hors bornes	Code 422, erreur de validation	Conforme

8.2.3 Portée et limites de cette validation

Ces résultats établissent que le contrat d'interface est correctement implémenté : les schémas sont cohérents, la validation rejette les entrées malformées, la sérialisation produit la structure attendue.

Ils n'établissent en revanche rien concernant la construction de l'image, le montage des volumes, la résolution des dépendances ou l'intégration avec le module réel. Cette distinction est essentielle et constitue le cœur de l'analyse critique du chapitre suivant : un test en isolation valide un composant, non un système.

8.3 Validation en conditions réelles

8.3.1 Conditions de réalisation

Le test de bout en bout a été réalisé par la responsable du déploiement lors de sa phase de validation, et non par l'auteur de la livraison. Cette validation a supposé la reconstruction complète de l'environnement de données sur sa machine : régénération de la transformation, exécution des tests qualité, validation du corpus, génération des vecteurs et construction du Vector Store.

La durée de cette reconstruction, environ deux cent soixante-dix minutes pour la seule génération des vecteurs sur son matériel, illustre le coût réel du diagnostic reporté en aval.

8.3.2 Défauts identifiés

Quatre défauts bloquants ont été mis en évidence, chacun empêchant le fonctionnement du service à un stade différent de la chaîne.

#	Défaut	Stade d'échec	Correction
1	Erreur de syntaxe dans la définition de l'image : un commentaire placé sur la même ligne qu'une commande d'installation interrompt l'interprétation de la chaîne	Construction de l'image	Commentaire déplacé sur sa propre ligne
2	Composition obsolète : un service de base vectorielle en réseau est déclaré alors que le module de récupération utilise une base embarquée ; le volume correspondant n'est pas monté	Démarrage du conteneur	Suppression du service inutile et montage du volume
3	Volume monté en lecture seule alors que la bibliothèque requiert un accès en écriture	Premier appel métier	Retrait de la restriction de lecture seule
4	Conflit de dépendances : deux bibliothèques exigent des versions mutuellement incompatibles d'une même dépendance transitive	Premier appel métier	Montée de version de la bibliothèque la plus contrainte

Tableau 21 — Défauts identifiés lors du déploiement et corrections apportées

8.3.3 Analyse des causes

Ces quatre défauts partagent une cause unique : l'absence d'exécution réelle du livrable avant transmission. Leur répartition selon le stade auquel ils se manifestent est instructive.

- **Défauts 1 et 2** — détectables par une simple commande de construction et de démarrage, sans aucune donnée réelle. Leur détection aurait demandé moins de cinq minutes.
- **Défauts 3 et 4** — détectables au premier appel métier réel, ce qui suppose la disponibilité du Vector Store. Leur détection aurait demandé la reconstruction de l'environnement.

La moitié des défauts était donc détectable pour un coût négligeable. Ce constat affaiblit considérablement la justification initiale, fondée sur le coût de régénération de l'environnement : ce coût ne s'appliquait qu'à la seconde moitié.

8.3.4 Validation finale après correction

<pre>curl -X POST http://localhost:8000/ask \ -H "Content-Type: application/json" \ -d '{"question": "What are the risk factors for \ cardiovascular disease?"}'</pre>
→ 200 OK
<pre>answer : réponse générée en français, adossée au corpus sources : 5 entrées (title, section, distance) latence : environ 67 secondes</pre>

Figure 10 — Appel réel validé de bout en bout

Après application des quatre corrections, la chaîne complète a été validée : récupération dans la base vectorielle, construction du prompt, génération par le service de langage, et retour d'une réponse accompagnée de ses sources.

8.4 Analyse de la latence

La latence mesurée constitue le principal enseignement quantitatif de la validation. Une décomposition indicative, fondée sur les métriques enregistrées par le module RAG dans le suivi d'expérimentations, permet d'en identifier les contributeurs.

Étape	Nature	Ordre de grandeur
Encodage de la question	Calcul local sur processeur	Faible
Recherche dans la base vectorielle	Lecture disque et calcul de similarité	Faible
Construction du prompt	Concaténation de chaînes	Négligeable
Appel au service de génération	Appel réseau distant, génération	Dominant
Enregistrement du suivi d'expérimentation	Écriture locale	Faible

Tableau 22 — Décomposition de la latence observée

Cette latence excède largement les attentes usuelles d'une interface synchrone : la plupart des clients HTTP appliquent par défaut un délai d'expiration inférieur à cette durée. Aucun délai d'expiration n'ayant par ailleurs été configuré côté serveur, une requête bloquée pourrait immobiliser le processus de travail indéfiniment. Ce point constitue la limite fonctionnelle la plus sérieuse de la livraison et fait l'objet d'une recommandation prioritaire au chapitre 10.

8.5 Livrables

Livrable	Description	État
api/main.py	Application et points d'entrée	Livré
api/schemas.py	Modèles de validation	Livré
api/Dockerfile	Définition de l'image	Livré, corrigé
api/requirements.txt	Dépendances de la couche d'exposition	Livré, corrigé
api/README.md	Documentation d'intégration	Livré
docker-compose.yml	Composition des services	Livré, corrigé

.env.example	Modèle de configuration fusionné	Livré
--------------	----------------------------------	-------

Tableau 23 — Livrables de la partie FastAPI & Docker

9. Intégration en Aval : CI/CD et Déploiement

Ce chapitre décrit la manière dont la livraison documentée a été reprise et exploitée par la responsable du déploiement. Il ne relève pas de la contribution individuelle, mais en constitue la validation ultime : un livrable d'industrialisation ne se juge pas sur sa forme, mais sur son exploitabilité effective par l'étape suivante.

9.1 Intégration continue

Un flux d'intégration continue a été mis en place, comportant deux tâches exécutées à chaque envoi de code sur le dépôt. La première régénère la transformation des données, exécute les huit tests qualité, lance la validation du corpus et vérifie la présence du jeu de données validé. La seconde construit l'image de l'API, conditionnée à la réussite de la première.

Cette seconde tâche revêt une importance particulière au regard de l'analyse du chapitre précédent : elle aurait détecté automatiquement le premier défaut, l'erreur de syntaxe dans la définition de l'image, dès l'envoi du code sur le dépôt. Ce constat illustre la valeur d'une automatisation qui ne dépend pas de la discipline individuelle.

Un contrôle automatisé ne prévient pas seulement les erreurs : il rend inutile la confiance dans la rigueur de chacun, ce qui constitue une propriété plus robuste dans une équipe de huit personnes travaillant en parallèle.

Enseignement tiré de la mise en place de l'intégration continue

9.2 Adaptation de la stratégie de déploiement

La plateforme de déploiement retenue par le module accepte une composition de services comme définition d'une pile applicative. La composition produite aurait donc pu être utilisée directement, ce qui constituait l'hypothèse de conception.

Les droits effectivement accordés sur la plateforme se sont révélés plus restreints que prévu : consultation du serveur et déploiement d'une pile préexistante, sans droit de création de dépôt ni accès à un terminal sur le serveur cible. Or la composition produite reposait sur deux éléments incompatibles avec ces restrictions : une construction de l'image sur le serveur cible, supposant l'accès au code source, et un montage de volume depuis le système de fichiers de l'hôte.

La responsable du déploiement a adapté la stratégie en construisant localement une image autonome intégrant le code et la base vectorielle, en la publiant sur un registre public, puis en ne référençant dans la plateforme que cette image prête à l'emploi. Cette adaptation a nécessité l'ajout, dans la définition de l'image, d'une instruction embarquant la base vectorielle.

Cette évolution mérite un commentaire : elle constitue un compromis assumé, l'intégration de données de plusieurs centaines de mégaoctets dans une image applicative contredisant le principe de séparation entre code et données. Elle était néanmoins la seule voie compatible avec les droits disponibles. Ce type d'arbitrage, imposé par des contraintes d'infrastructure plutôt que par des considérations techniques, est fréquent en situation réelle.

9.3 État final de la solution

Volet	État	Commentaire
Validation du build applicatif	Terminé	Quatre défauts corrigés, réponse réelle obtenue
Intégration continue	Fonctionnelle	Deux tâches vertes à chaque envoi

Déploiement de la pile	Terminé	Conteneurs actifs sur le serveur cible
Vérification de l'accès externe	Bloquée	Restriction réseau ou pare-feu, hors périmètre technique
Monitoring et alertes	Non réalisé	Dernière composante du niveau de maturité 2

Tableau 24 — État global de la solution déployée

Le déploiement applicatif est confirmé par la plateforme, les conteneurs étant actifs et le port correctement associé. L'impossibilité de valider l'accès depuis l'extérieur relève d'une limitation d'infrastructure ou de droits, et non d'un défaut de la solution elle-même.

10. Bilan Critique

10.1 Apports de la démarche

10.1.1 La lecture préalable des passations

La phase d'analyse documentaire, qui pouvait apparaître comme une formalité, s'est révélée la plus rentable de la mission. Trois décisions de conception en découlent directement, et aucune n'aurait été prise correctement par simple intuition.

- La politique de codes de statut, fondée sur le constat que les défaillances du service de génération sont absorbées en amont. Une conception par défaut aurait produit un code d'erreur serveur sur un cas parfaitement nominal.
- La déclaration du serveur de suivi d'expérimentations comme dépendance obligatoire, et non comme outil de développement optionnel.
- Le caractère optionnel des champs de métadonnées dans le modèle de source, découlant de la tolérance documentée par la responsable du contrôle qualité sur les chunks sans titre de section explicite.

10.1.2 La stabilité du contrat d'interface

Le choix de calquer strictement les schémas sur le format réel retourné par la fonction sous-jacente, sans couche de transformation intermédiaire, s'est révélé judicieux. Les quatre corrections apportées lors du déploiement portaient exclusivement sur la configuration de l'environnement : aucune n'a nécessité de modification du code de l'API.

10.1.3 La cohérence technologique

La réutilisation de Pydantic, déjà adopté par l'équipe au Sprint 2, a évité l'introduction d'une dépendance conceptuelle supplémentaire. Cet argument, qui relève de l'économie cognitive collective plutôt que de la performance technique, mérite d'être valorisé : dans un projet à huit contributeurs, la réduction du nombre d'outils distincts constitue en soi une forme de qualité.

10.2 Faiblesse principale : la validation reportée

10.2.1 Constat

La principale faiblesse de la mission doit être énoncée sans atténuation : le livrable a été transmis sans avoir jamais été exécuté en conditions réelles. Les tests menés validaient le contrat d'interface, mais ne pouvaient structurellement révéler ni un défaut de construction d'image, ni un problème de montage de volume, ni un conflit de dépendances.

La définition de fin de mission stipulait que la responsable du déploiement devait pouvoir déployer l'ensemble sans modification majeure. Quatre corrections ayant été nécessaires, ce critère n'était pas satisfait au moment de la transmission.

10.2.2 Examen de la justification avancée

La justification initiale reposait sur le coût de régénération de l'environnement, estimé entre quatre-vingts et quatre-vingt-dix minutes. Cette justification était factuellement exacte mais logiquement insuffisante, pour deux raisons.

Première raison. La moitié des défauts, ceux relatifs à la construction de l'image et à la définition des services, était détectable sans aucune donnée réelle, par une simple commande de construction et de

démarrage. Le coût de cette vérification était de quelques minutes. La justification par le coût de l'environnement ne couvrait donc que la moitié du problème.

Seconde raison. Le coût évité était très inférieur au coût transféré. La responsable du déploiement a dû régénérer l'environnement complet, avec une durée de génération des vecteurs d'environ deux cent soixante-dix minutes sur son matériel, puis diagnostiquer quatre défauts successifs sans disposer du contexte de développement ni de l'historique des décisions. Le temps total consommé en aval excède largement celui qui aurait été nécessaire en amont.

10.2.3 Analyse au regard de la littérature

Cette situation illustre précisément le mécanisme décrit par Sculley et ses coauteurs sous le terme de dette technique cachée [5]. Le report d'une vérification constitue un emprunt : il libère du temps immédiatement, mais l'intérêt s'accumule et se paie au moment du diagnostic, avec une majoration liée à la perte de contexte.

Breck et ses coauteurs observent par ailleurs que les tests d'infrastructure sont systématiquement sous-investis par rapport aux tests de modèle [13]. Les quatre défauts relèvent tous de l'infrastructure : aucun ne concerne la qualité des réponses, la pertinence de la récupération ou la validité du contrat d'interface. Cette répartition confirme empiriquement l'observation des auteurs.

Dans une chaîne de dépendances, un critère de fin de mission n'est pas une formalité administrative mais une garantie contractuelle envers le membre suivant. Le coût d'un défaut croît à chaque étape franchie, car il est diagnostiqué par quelqu'un qui n'a ni le contexte, ni l'environnement, ni l'historique des décisions de son auteur.

Enseignement principal de la mission

10.3 Limites fonctionnelles de la livraison

Limite	Conséquence	Criticité
Latence d'environ 67 secondes sans délai d'expiration configuré	La plupart des clients HTTP expireraient avant réception	Élevée
Absence de point d'exposition de métriques	Le monitoring en aval n'a pas pu être mis en place	Élevée
Absence d'authentification sur le point métier	Exposition publique impossible en l'état	Moyenne
Processus de travail unique	Débit limité à une requête simultanée	Moyenne
Modèle d'encodage téléchargé au premier appel	Démarrage à froid allongé, dépendance réseau	Moyenne
Orchestrateur absent de la composition	Le pipeline de données n'est pas conteneurisé avec le service	Faible

Tableau 25 — Synthèse des limites identifiées

10.4 Ce qui aurait dû être fait différemment

Trois modifications de la démarche auraient prévenu l'essentiel des difficultés rencontrées.

1. **Vérifier la construction de l'image dès sa rédaction.** Une commande de construction exécutée immédiatement après l'écriture de la définition aurait révélé l'erreur de syntaxe en moins d'une minute, sans nécessiter aucune donnée.
2. **Relire la composition à la lumière du README du module amont.** La nature embarquée de la base vectorielle y était explicitement documentée. Le service réseau déclaré à tort provenait d'une hypothèse formée avant la lecture, jamais révisée ensuite.

3. **Négocier une régénération partagée de l'environnement.** Plusieurs membres ont régénéré indépendamment le même environnement de plusieurs centaines de mégaoctets. Une mise en commun, par archive partagée ou volume préconstruit, aurait supprimé l'obstacle invoqué pour justifier l'absence de test.

11. Perspectives d'Amélioration

11.1 Traitement de la latence

La latence observée constitue la limite la plus sérieuse. Trois approches sont envisageables, par ordre de complexité croissante.

Configuration d'un délai d'expiration. Mesure minimale, à mettre en œuvre en priorité : un délai côté serveur évite qu'une requête bloquée n'immobilise durablement le processus de travail.

Traitement asynchrone. Le point métier retournerait immédiatement un identifiant de tâche, la réponse étant récupérée ultérieurement par consultation. Cette approche découple la durée de traitement de la durée de la connexion HTTP et constitue le modèle usuel pour les traitements longs.

Diffusion progressive de la réponse. La génération étant produite jeton par jeton par le modèle de langage, une diffusion en flux permettrait d'afficher la réponse au fur et à mesure. La latence perçue par l'utilisateur serait considérablement réduite, même si la durée totale demeurerait inchangée.

11.2 Observabilité

L'exposition d'un point de métriques au format attendu par un système de collecte permettrait la mise en place du monitoring, actuellement absent. Les indicateurs pertinents comprendraient le nombre de requêtes par unité de temps, la distribution des latences, le taux d'erreur, ainsi que des indicateurs spécifiques au RAG tels que la distance moyenne de récupération, dont une dérive signalerait un désalignement entre les questions posées et le contenu du corpus.

Cette instrumentation rejoint directement les recommandations de Breck et ses coauteurs concernant la surveillance des systèmes en production [13].

11.3 Optimisation du démarrage

Le préchargement du modèle d'encodage dans l'image supprimerait la dépendance réseau au démarrage et réduirait la durée du démarrage à froid. Cette modification augmenterait la taille de l'image mais améliorerait la prévisibilité du déploiement, propriété généralement préférable en production.

11.4 Sécurisation

Aucun mécanisme d'authentification n'est en place. Avant toute exposition publique, la mise en place d'une authentification par clé, assortie d'une limitation du débit de requêtes, constitue un prérequis. Cette dernière protège également contre l'épuisement du quota du service de génération, dont l'accès gratuit est intrinsèquement limité.

11.5 Évaluation quantitative du système

L'évaluation du système demeure qualitative, limite reconnue par les responsables des étapes de vectorisation et de construction de la chaîne RAG. L'adoption d'un cadre tel que celui proposé par Es et ses coauteurs [18] permettrait de mesurer la fidélité des réponses aux sources récupérées, leur pertinence à la question posée, et la pertinence des documents récupérés, sans nécessiter de réponses de référence annotées.

Une telle instrumentation, exposée au travers de la couche applicative, permettrait en outre de suivre l'évolution de ces indicateurs dans le temps et de détecter une dégradation liée à une évolution du corpus.

11.6 Amélioration de la récupération

Deux pistes ressortent de l'état de l'art présenté au chapitre 2. Une approche hybride combinant recherche lexicale et recherche dense [7][8] améliorerait vraisemblablement la récupération sur les termes techniques rares, fréquents en littérature biomédicale. Un réordonnancement des résultats par un modèle d'appariement croisé, mentionné dans la revue de Gao et ses coauteurs [4], améliorerait la précision des premiers résultats au prix d'une latence supplémentaire.

Ces deux pistes relèvent toutefois des étapes amont du pipeline et supposeraient une coordination avec les responsables concernés.

12. Conclusion

Le projet PipelineForge a abouti à une solution fonctionnelle de bout en bout, depuis l'ingestion de sept cents articles biomédicaux jusqu'à un service conteneurisé déployé sur une plateforme d'orchestration. Le corpus, segmenté en 32 146 chunks, validé par une suite de tests automatisés, vectorisé et indexé dans une base vectorielle, alimente une chaîne de génération produisant des réponses systématiquement adossées à des sources identifiables.

La contribution documentée dans ce rapport a consisté à franchir l'étape séparant un module de recherche d'un service exploitable. Cette transition, souvent perçue comme périphérique au regard du travail de modélisation, concentre en réalité une part importante des difficultés pratiques des systèmes d'apprentissage automatique, conformément à l'analyse de Sculley et ses coauteurs [5].

Sur le plan fonctionnel, l'objectif est atteint : l'interface expose la chaîne RAG, valide les échanges, applique une politique de codes de statut cohérente avec le comportement réel des composants amont, et s'exécute dans un environnement conteneurisé reproductible. Après correction, un appel réel produit une réponse sourcée exploitable.

Sur le plan méthodologique, le bilan appelle davantage de nuance. Quatre défauts bloquants ont été identifiés en aval, tous imputables à l'absence d'une validation en conditions réelles avant transmission. Ce constat n'est pas anecdotique : il illustre un mécanisme général, par lequel le report d'une vérification transfère un coût majoré à l'étape suivante, diagnostiqué par une personne privée du contexte de développement.

L'enseignement principal de cette mission tient en une formulation : en industrialisation, un livrable qui n'a jamais été exécuté dans son environnement cible n'est pas un livrable, mais une hypothèse. La distinction entre un composant qui fonctionne et un système qui fonctionne constitue, en définitive, l'objet même de la discipline MLOps.

Bibliographie

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30.
- [2] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12), 1–38.
- [3] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33.
- [4] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv preprint arXiv:2312.10997*.
- [5] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden Technical Debt in Machine Learning Systems. *Advances in Neural Information Processing Systems (NeurIPS)*, 28.
- [6] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186.
- [7] Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- [8] Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *Proceedings of EMNLP 2020*, 6769–6781.
- [9] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP-IJCNLP 2019*, 3982–3992.
- [10] Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2023). C-Pack: Packaged Resources To Advance General Chinese Embedding. *arXiv preprint arXiv:2309.07597*.
- [11] Johnson, J., Douze, M., & Jégou, H. (2021). Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547.
- [12] Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 824–836.
- [13] Breck, E., Cai, S., Nielsen, E., Salib, M., & Sculley, D. (2017). The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction. *Proceedings of IEEE Big Data 2017*, 1123–1132.
- [14] Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., Nagappan, N., Nushi, B., & Zimmermann, T. (2019). Software Engineering for Machine Learning: A Case Study. *Proceedings of ICSE-SEIP 2019*, 291–300.
- [15] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Thèse de doctorat, University of California, Irvine.
- [16] Merkel, D. (2014). Docker: Lightweight Linux Containers for Consistent Development and Deployment. *Linux Journal*, 2014(239).
- [17] Boettiger, C. (2015). An Introduction to Docker for Reproducible Research. *ACM SIGOPS Operating Systems Review*, 49(1), 71–79.
- [18] Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2024). RAGAS: Automated Evaluation of Retrieval Augmented Generation. *Proceedings of EACL 2024, System Demonstrations*.
- [19] Chen, J., Lin, H., Han, X., & Sun, L. (2024). Benchmarking Large Language Models in Retrieval-Augmented Generation. *Proceedings of AAAI 2024*, 38(16), 17754–17762.
- [20] Documentation FastAPI (2024). FastAPI — High performance, easy to learn, fast to code, ready for production. <https://fastapi.tiangolo.com/>

- [21] Documentation Pydantic (2024). Pydantic — Data validation using Python type hints.
<https://docs.pydantic.dev/>
- [22] Documentation Docker (2024). Docker Compose — Define and run multi-container applications.
<https://docs.docker.com/compose/>
- [23] Documentation MLflow (2024). MLflow Tracking — Recording and querying experiments.
<https://mlflow.org/docs/latest/tracking.html>
- [24] Documentation Chroma (2024). Chroma — The AI-native open-source embedding database.
<https://docs.trychroma.com/>
- [25] National Library of Medicine (2024). PubMed Central Open Access Subset.
<https://www.ncbi.nlm.nih.gov/pmc/tools/openftlist/>

Annexe A — Code Source de l'Interface Applicative

A.1 main.py

```
"""
main.py – API FastAPI (Chbab Brahim, Sprint 4)
Expose le module RAG d'Hafsa Elhilali via HTTP.
"""
import time
import logging

from fastapi import FastAPI, HTTPException
from schemas import AskRequest, AskResponse, HealthResponse, Source

# Le dossier rag/ est copié dans l'image et ajouté au PYTHONPATH
from rag_chain import answer_question

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("pipelineforge-api")

app = FastAPI(
    title="PipelineForge – Assistant scientifique documentaire",
    description="API exposant la chaîne RAG sur le corpus biomédical.",
    version="1.0.0",
)

@app.get("/health", response_model=HealthResponse)
def health():
    """Vérifie que le service est en ligne."""
    return HealthResponse(status="ok")

@app.post("/ask", response_model=AskResponse)
def ask(request: AskRequest):
    start = time.time()
    try:
        result = answer_question(
            question=request.question,
            top_k=request.top_k,
            where=request.where,
        )
    except Exception as e:
        logger.exception("Échec de answer_question()")
        raise HTTPException(
            status_code=500,
            detail=f"Erreur interne du pipeline RAG : {e}",
        )

    return AskResponse(
        answer=result["answer"],
        sources=[Source(**s) for s in result["sources"]],
        latency_ms=(time.time() - start) * 1000,
    )
```

A.2 requirements.txt

```
fastapi==0.115.0
```

```
uvicorn[standard]==0.30.6  
pydantic==2.9.2  
mlflow>=2.19          # >=2.19 requis : conflit protobuf avec chromadb
```

Annexe B — Procédure de Reprise

Cette annexe décrit la procédure permettant à un tiers de reprendre et d'exécuter la solution.

B.1 Prérequis

- Docker et Docker Compose installés et fonctionnels sur la machine hôte.
- Le dépôt cloné, positionné sur la branche contenant la couche d'exposition.
- Le Vector Store généré localement — environ 80 à 90 minutes — le répertoire n'étant pas versionné.
- Une clé d'accès valide auprès du service de génération.

B.2 Séquence d'exécution

```
# 1. Régénérer la chaîne de données (si absente)
python orchestration/pipeline.py

# 2. Générer les vecteurs puis le Vector Store
python embeddings/build_embeddings.py      # ~76-90 min
python embeddings/build_vectorstore.py    # ~1 min

# 3. Créer le fichier de configuration
cp .env.example .env
#   puis renseigner GROQ_API_KEY

# 4. Construire et démarrer la solution
docker compose up --build

# 5. Vérifier la disponibilité
curl http://localhost:8000/health

# 6. Effectuer un appel métier
curl -X POST http://localhost:8000/ask \
  -H "Content-Type: application/json" \
  -d '{"question": "How is diabetes diagnosed?", "top_k": 5}'
```

B.3 Points de vigilance

Symptôme	Cause probable	Résolution
Échec de construction de l'image	Commentaire mal placé ou dépendance manquante	Vérifier la définition de l'image ligne par ligne
Erreur de base en lecture seule	Volume monté avec restriction d'écriture	Retirer la restriction dans la composition
Conflit de versions au démarrage	Dépendances transitives incompatibles	Vérifier les contraintes de version déclarées
Premier appel très lent	Téléchargement du modèle d'encodage	Comportement attendu, prévoir l'accès réseau
Absence de réponse après 60 secondes	Latence nominale du service de génération	Augmenter le délai d'expiration du client